Low Power VLSI Architectures for One Bit Transformation Based Fast Motion Estimation

Sumit K Chatterjee and Indrajit Chakrabarti

Abstract — *In the present paper, architectures implementing fixed block size and variable block size (VBS) motion estimation (ME) algorithms on one bit transformed (1-BT) image frames have been presented. The proposed architectures perform ME by applying diamond search (DS) algorithm on 1-BT image frames. The 1-BT based ME is usually performed by applying full search (FS) algorithm. Our simulation results reveal that the application of DS on 1-BT based ME can significantly reduce the computational complexity which is observed in FS based 1-BT ME at a tolerable degradation in the quality. In terms of latency, the architectures have been shown to be superior to several other 1-BT ME architectures. The presented DS based 1-BT ME architectures have succeeded in reducing the minimum clock frequency required to process a video sequence with a given frame size and frame rate, which in turn reduces the overall power consumption compared with other ME architectures. In particular, for processing SDTV sequences (1280×720 @ 30 fps), the power consumption by the proposed VBS ME architecture is reduced by at least 41% compared to the other 1-BT based ME architectures available in literature. The proposed architectures are therefore, considered suitable for low-power portable video applications typically operated by battery power*[1].

Index Terms — One bit transformation, binary motion estimation architecture, data reuse, diamond search algorithm, low power design, variable block size.

I. INTRODUCTION

Due to the stringent requirements of the real time video playback systems, video coding is the most essential part of any visual application. Furthermore, due to the limited channel and storage capacity, these applications require a very high compression ratio as well. Motion estimation (ME), which is the most essential part of any video coding technique, exploits and tries to minimize the temporal redundancy present between successive frames. ME, which is computationally intensive, involves about 80% of the total computational power of the encoder [1]. Block matching motion estimation (BMME) is one of the most efficient and popular techniques to remove the temporal redundancy present between successive frames [2]. In this method, video frame is partitioned into blocks of 16×16 pixels known as macro-blocks (MBs). For each MB in the current frame, one searches for the best matched block (within a search range) in a previously available frame known as the reference frame. Motion vector (MV) is defined as the displacement between the current block position and the best matched one in the reference frame. This process is repeated until MV is found for all the blocks in the current frame. In order to improve the coding efficiency, variable blocks size (VBS) ME has been adopted in modern coding standards like H.264.

The matching criterion of the BMME method has a direct impact on the coding efficiency and computational complexity. Many matching criteria have been proposed in literature e.g., mean squared error, sum of absolute differences (SAD), pel difference classification etc. [3]. Whatever may be the matching criterion, evaluation of the matching criterion on pixels with 8 bits/pixel representation requires a huge amount of computation. The computational load can be reduced to a great extent by representing the pixels with a reduced number of bits. As proposed in [4], an image frame with 8 bits/pixel representation is first converted into a binary frame with 1 bit/pixel representation. ME is then carried out on these binary image frames. Boolean exclusive OR (XOR) operation is used to find the number of non-matching points (NNMP). In this method, NNMP is regarded as the matching criterion in place of the conventional SAD.

In literature, many one-bit transformation (1-BT) kernels are available that convert an image with 8 bits/pixel representation into a binary image with 1 bit/pixel representation. The first 1-BT kernel was proposed by Natarajan et al. [4]. A multiplication free one-bit transformation (MF-1BT) kernel has been proposed in [5]. Binary ME with an early termination scheme have been proposed in [6]. Two bit transformation (2-BT) has been proposed in [7] for an enhanced accuracy in ME. The constrained one bit transformation (C-1BT) has been proposed in [8] for a better performance than 1-BT, but at the same time with a reduced complexity than 2-BT.

The implementation of 1-BT based ME on hardware has been proposed for the first time by Natarajan et al. [4]. A high performance VBS ME architecture for MF-1BT has been proposed in [9]. In [10], MF-1BT and C-1BT ME architectures for fixed block size (FBS) are presented. Both the architectures [4] and [10] are based on 16 processing elements (PEs).

The application of fast search algorithms like diamond search (DS) on 1-BT frames can further reduce the computational load to a great extent as compared to applying full search (FS) on 1-BT frames. It has been shown in the present paper that the combination of 1-BT with DS results in a very small degradation in the peak signal to noise ratio (PSNR) as compared to FS.

In the present paper, we have presented an in depth analysis of performing ME on 1-BT binary frames by applying DS

[1] Sumit K Chatterjee and Indrajit Chakrabarti are currently with the Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology, Kharagpur-721302, WB; India (e-mail: {skchatterjee, indrajit}@ece.iitkgp.ernet.in).

Contributed Paper
Manuscript received 10/08/10
Current version published 12/23/10
Electronic version published 12/30/10 0098 3063/10/$20.00 © 2010 IEEE

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

algorithm and proposed architectures for 1-BT based FBS and VBS motion estimation. In the proposed architectures, the pixels from the current block (CB) are read only once from the external memory and are stored into the local memories of the ME hardware. Thereafter, the pixels from the CB are supplied to the appropriate PEs periodically. The overlap between the neighboring search locations for DS is also exploited in a novel way to reduce the number of external memory accesses. The proposed architectures are faster than a recently reported 1-BT based ME architecture [9].

The present paper is organized as follows. Section II presents a brief introduction to the fast binary ME achieved by combining 1-BT with DS algorithm. The novel data flow analysis for the DS algorithm has been provided in Section III. Section IV presents the proposed FBS 1-BT ME architecture in detail. The VBS ME architecture has been presented in section V. Relevant results dealing with synthesis of the proposed architectures may be found in section VI. Conclusions are finally drawn in section VII.

II. ONE BIT TRANSFORMATION AND DIAMOND SEARCH ALGORITHM

A. One Bit Transformation Based ME

Before applying 1-BT based ME, the original image frames having 8 bits/pixel representation is converted into binary image frames with 1 bit/pixel representation. This is done by filtering the original image frame by a multi band-pass filter. The filtered image is then compared with the original image to obtain the binary image. In the original work by Natarajan et al. [4], the kernel used for filtering was having 25 non-zero elements and required expensive floating point multiplications. MF-1BT has been proposed in [5] in which the complex floating point multiplications were replaced by simple shifting operations. This new kernel '*K*' in matrix form has been shown in (1).

As in this kernel the normalization factor is a power of 2, the filtering can now be performed by mere shifting without any expensive multiplication operation. The original frame F is filtered by convolving it with *K* and the filtered frame $\hat{F}$ is obtained and then one-bit image frames are constructed as:

$$K = \frac{1}{16}\begin{bmatrix}
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&1&0&0&0&0&0&1&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&1&0&0&0&0&0&1&0&0&0&0&0&1&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
1&0&0&0&0&0&1&0&0&0&0&0&1&0&0&0&0&0&1\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&1&0&0&0&0&0&1&0&0&0&0&0&1&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&1&0&0&0&0&0&1&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0&0\\
0&0&0&0&0&0&0&0&0&1&0&0&0&0&0&0&0&0
\end{bmatrix} \quad (1)$$

$$B(i,j) = \begin{cases} 1, \text{ if } F(i,j) \geq \hat{F}(i,j) \\ 0, \text{ otherwise.} \end{cases} \quad (2)$$

Here, *i* and *j* are the spatial coordinates of the pixel. The foregoing process by which an original frame with 8 bits/pixel representation is converted into a binary frame with 1 bit/pixel representation is known as one-bit transformation. After this operation, the number of non-matching points (*NNMP*) at any point (*m,n*) for a MB of size *N*×*N* is found as:

$$NNMP(m,n) = \sum_{i=0}^{N-1}\sum_{j=0}^{N-1} B^{t}(i,j) \oplus B^{t-1}(i+m, j+n) \quad (3)$$

where, $-s \leq m,n \leq s$.

Here, '*s*' is the maximum search range and $\oplus$ denotes XOR operation. Also, B^t and B^{t-1} represent the current and the reference 1-BT frames respectively.

B. Diamond Search Based 1-BT ME

In DS algorithm [11], the search pattern forms a diamond like shape as shown in Fig. 1. Also, there is no limit on the number of steps that may be involved in the algorithm. DS uses two fixed search patterns, where one is the large diamond search pattern (LDSP) and the other is the small diamond search pattern (SDSP). The search starts with the LDSP with its center of search located at the origin. The matching criterion is evaluated for all the 9 points of LDSP and if the best match is found at the center, then the search pattern is switched to SDSP; else, another LDSP is selected with the new search center pointing to the location where the best match is found. This process is repeated until the best match is found at the center of the LDSP, after which the search pattern is switched to SDSP where the matching criterion is evaluated for 4 points around the center. The final MV is then found from the point giving the best match at this step.

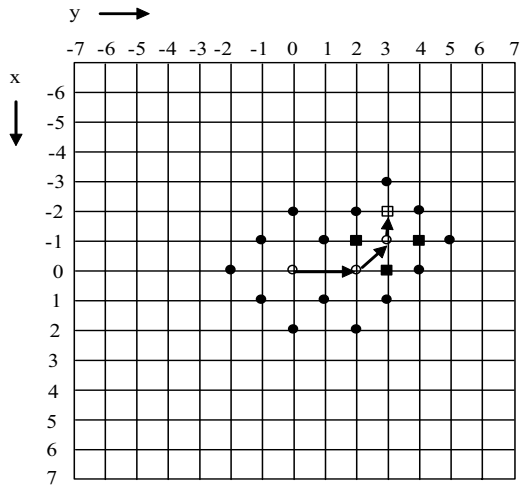


Fig. 1. An example illustrating the search strategy of DS algorithm

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

Fig. 1 demonstrates pictorially the steps required to find the motion vector (-2,+3). The solid spheres indicate the location of the search points for LDSP. The solid squares indicate the location of the search points for SDSP. The locations of the minimum SAD points are indicated by transparent sphere (square) for LDSP (SDSP).

As DS algorithm is highly center biased, it is very fast as compared to FS algorithm. On the other hand, unlike other fast search techniques such as the new three step search, where the number of search steps are fixed, in DS the number of search steps are not fixed and thus its performance is very much close to that of FS in terms of the PSNR [12].

The work on development of ME architecture as reported in the present paper is based on a novel combination of 1-BT and DS. ME is usually performed on 1-BT frames by full search only [9], [10]. In the present work, several fast search techniques (e.g. three step search TSS), new three step search (NTSS), four step search (4-SS), and diamond search (DS)) have been applied on 1-BT frames. The performance of fast ME on 1-BT frames is evaluated based on PSNR and the average number of search points.

In Table I, the PSNR values obtained by applying different fast search algorithms on 1-BT frames have been shown for different benchmark video sequences. All the sequences are in CIF (352×288) format, and each of the sequences contains 300 frames. The search range is taken as [-16,16] along both the axes. The average number of search points required to generate a MV can be regarded as a metric to measure the computational complexity for block matching. The average number of search points for different search algorithms on different video sequences have been shown in Table II.

It can be observed from Table I that the combination of DS and 1-BT displays PSNR performance similar to the application of FS on 1-BT in most sequences with less than 0.21 dB degradation except for the sequences with complex motion like Foreman and the Coastguard for which the performance degrades by 0.31 dB and 0.59 dB respectively.

On the other hand, it can be seen from Table II that DS based 1-BT ME always provides faster results than other fast search techniques. Taking all these observations into account, it can be inferred that applying DS on 1-BT frames provides almost the same performance as that of the FS based 1-BT ME, but at much lower computational complexity.

TABLE I
PERFORMANCE COMPARISON IN TERMS OF PSNR (dB)

Video Sequence		Full Search	TSS	NTSS	4-SS	DS
Foreman	Max.	32.208	29.678	31.816	31.078	31.960
	Min.	28.106	25.900	27.652	27.965	27.899
	Avg.	30.766	28.982	29.651	30.031	30.458
Hall	Max.	34.388	31.631	33.599	34.163	34.478
Monitor	Min.	30.423	28.161	29.890	30.318	30.330
	Avg.	33.755	31.129	32.535	33.160	33.675
Football	Max.	23.262	22.435	22.160	22.616	22.966
	Min.	17.004	16.189	16.891	16.868	16.969
	Avg.	21.639	19.816	20.953	21.101	21.448
Tennis	Max.	30.465	28.899	29.997	30.108	30.169
	Min.	22.307	21.115	21.169	21.076	21.987
	Avg.	28.387	26.158	27.586	27.618	28.178
Coastguard	Max.	31.672	29.178	30.661	31.306	31.217
	Min.	23.191	21.806	22.898	22.165	22.766
	Avg.	29.512	27.638	28.760	28.898	28.922

TABLE II
AVERAGE NUMBER OF SEARCH POINTS PER MV GENERATION. THE NUMBER OF SEARCH POINTS FOR TSS AND FS ARE FIXED, NAMELY 25 AND 1089 RESPECTIVELY FOR A SEARCH RANGE OF [-16,16]

Video Sequence	NTSS	4-SS	DS
Foreman	30.15	26.80	24.00
Hall Monitor	22.68	20.43	13.30
Football	22.56	21.58	17.70
Tennis	20.59	21.18	19.96
Coastguard	17.50	16.53	13.85

III. DATA FLOW ANALYSIS FOR DS ALGORITHM

Due to the regularity in data-flow, FS motion estimation (FSME) is generally preferred from the implementation point of view. However, due to its high computational power requirements, many fast but sub-optimal search algorithms have been proposed. These fast search algorithms can reduce the computational power by up to 60%, but as a side effect introduce irregular data flow [13]-[15]. In our experiment, it has been found that DS based 1-BT ME provides acceptable quality at much lower complexity than FS based 1-BT ME. Therefore, DS algorithm is one of the most preferred fast search algorithms.

On the other hand, as DS introduces irregular data flow, the implementation of DS in hardware imposes considerable challenge for a VLSI designer. The search points for DS algorithm are arranged in a diamond like shape. This makes memory access mechanism for DS to be completely different from the FSME. The technique by which the data reuse is done for FS cannot be applied to DS in the same way.

It has been shown in [17], [18] that the memory access is the most costly operation in ME hardware. Therefore, it is possible to reduce the overall power consumption of the ME hardware by reducing the number of external memory accesses. In the present section, a novel data-flow has been presented, which reduces the power consumption by maximizing the data reuse.

In Fig. 2, the center of search is represented by location '4', and all the eight search locations around the center are represented by indexes starting from '0' to '8'. In order to understand the data overlap among different search locations, let us consider the size of the block as 4×4, and the center of search is located at pixel coordinates (2,2). Now, with reference to Fig. 2 and Fig. 3, location '0' corresponds to the pixel coordinates (0,2), location '1' corresponds to the pixel coordinates (1,1), and so on.

In order to reduce the processing time for 1-BT based ME, it is desirable to process an entire row of data from the current block as well as the search block simultaneously ([4], [10]). In that case, one has to access an entire row of data from both of the blocks simultaneously. Now, with reference to Fig. 2 and Fig. 3, one can observe that the search pixels for the entire first

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

row corresponding to the location '0' in Fig. 2 have the pixel coordinates (0,2), (0,3), (0,4) and (0,5) in Fig. 3. It is also obvious from Fig. 3 that the first row of search pixels corresponding to the location '0' can be obtained from the same row, namely 'Row0'. Similarly, the search pixels for the entire first row corresponding to the search locations '1' and '2' can be obtained from the same row, namely 'Row1'. In a similar way, 'Row2' provides the search pixels for the entire first row corresponding to the search locations '3', '4', and '5'. Similarly, the first row of search pixels for the search locations '6' and '7' are obtained from Row3 and lastly, the first row search pixels for the search location '8' are obtained from Row4. Thus, by exploiting the overlap of the search pixels for different search locations, one can reduce the number of memory accesses to 5 (Row0 to Row4) instead of 9 as would have been required for a straightforward implementation.

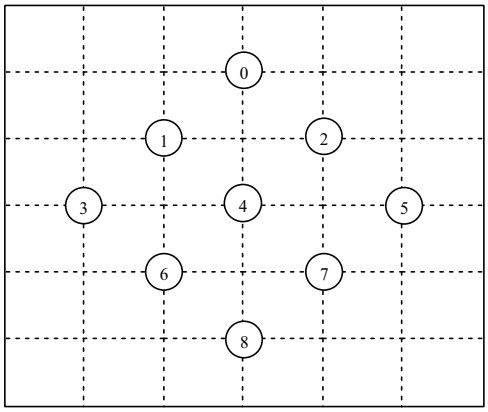


Fig. 2. Search locations for DS algorithm. The center of search is denoted by '4'.

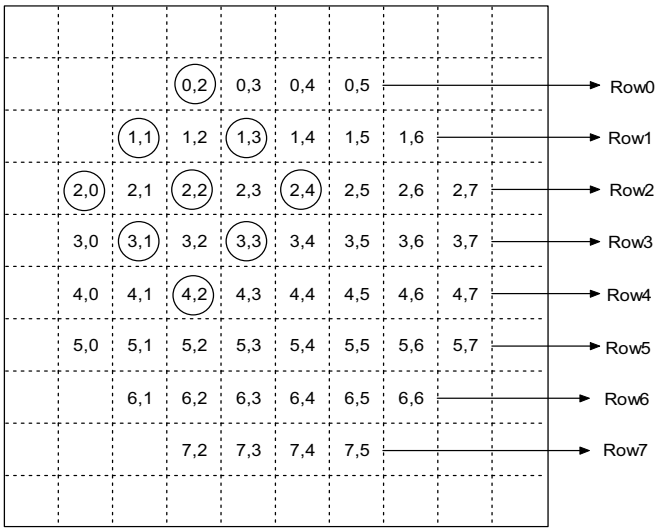


Fig. 3. Data flow example for DS with pixel coordinate (2,2) as the center of search

It may be noted that the second row corresponding to the search location '0' in Fig. 2 has the pixel coordinates (1,2), (1,3), (1,4), and (1,5) in Fig. 3. Thus, the second row of data corresponding to the search location '1' can be obtained from 'Row1' (as shown in Fig. 3) which has already been read in the last step. Similar is the case for all the search locations except for the location '8', for which an entire new row (i.e. Row5) has to be read. Thus, only a new row of data has to be read in the subsequent steps of ME. Therefore, by exploiting the overlaps of the search pixels for different search locations the number of memory accesses for search pixels can he reduced down to 8 (5+1+1+1), whereas 36 (9×4) memory accesses (corresponding to the fact that there exist 9 search locations and the block size is 4×4 as mentioned earlier in the present section) would have been required for a straightforward implementation of DS algorithm.

IV. PROPOSED VLSI ARCHITECTURE FOR 1-BT BASED FIXED BLOCK SIZE MOTION ESTIMATION

The block diagram of the proposed architecture for performing FBS ME on binary frames by applying DS has been shown in Fig. 4. The architecture includes 8 RAMs for storing all the pixels from the search window (SW), a register array for storing the current pixels, a register array for storing the search pixels termed as the 'search register array', a data selector array, nine PEs, a comparator and a process control unit. In the proposed scheme, the evaluations of the matching criterion for all the 9 search locations shown in Fig. 2 are done in parallel. As there are 9 search locations for DS algorithm, a total of 9 PEs are used.

Before the start of ME process for each MB, the search pixels from the entire SW are loaded into the internal memory of the ME hardware which consists of 8 RAMs. The CB pixels are also loaded into the register array for storing the current pixels. In the first step of ME, the center of search is chosen as the origin and the search pixels required for computing the matching criteria at the center are loaded into the search register array from 8 RAMs. The search register array then provides proper search pixels to all the PEs. All the PEs start the evaluation of the matching criterion (NNMP) for all the 9 search locations simultaneously. The computed values of the NNMPs are sent to the comparator. The comparator finds the minimum NNMP value as well as the corresponding location and sends it to the process control unit. The process control unit controls the entire process of ME by sending proper control signals to all the hardware modules.

The internal structure of each of the modules of the proposed hardware is provided in the following subsections.

A. Processing Element

The architecture of the PE is shown in Fig. 5. The PE possesses two input ports namely, C and S for reading two 16-bit vectors from the CB and the SW respectively. There are two 8-bit XOR arrays in the PE. One of the XOR arrays operates on the eight most significant bits of the 16-bit vectors C and S, and the other operates on the remaining eight least significant bits. The number of ones as a result of the XOR operation is obtained by using two look-up tables (LUTs) with 2^8 entries. The outputs of the LUTs are then applied to a 4-bit adder. The output of the adder is then applied at the input of the accumulator. The output of the accumulator provides the value of the NNMP for a particular location after 16 clock cycles for a MB of size 16×16.

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

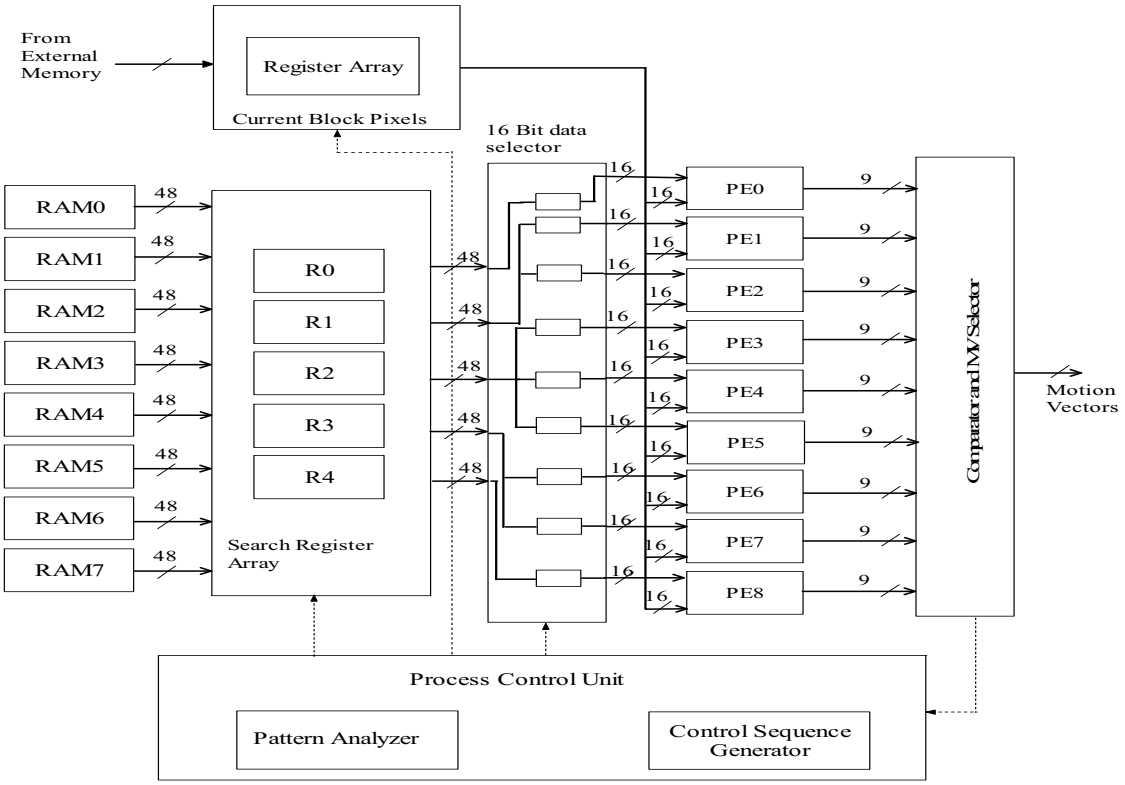


Fig. 4. Overall VLSI architecture for the proposed FBS ME

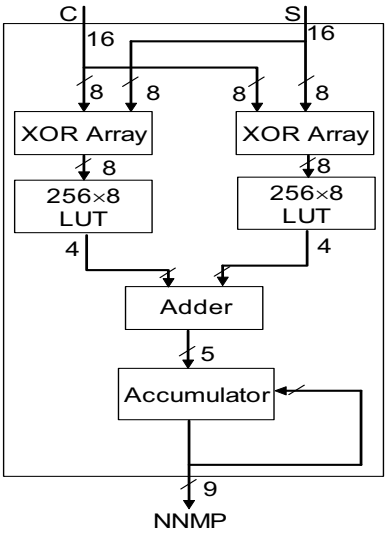


Fig. 5. PE architecture for 1-BT FBS ME

B. Memory Interleaving

In order to provide the required search pixels to all the PEs simultaneously, memory interleaving technique similar to [19] is exploited. The search pixels from the SW are interleaved among 8 RAMs (indexed from 0 to 7). For a single MB of size 16×16 pixels and for a search range of [-16,16] pixels, the size of the SW will be 48×48 pixels. Here, since each pixel is represented by one bit, the actual size of the SW would be 48×48 bits. The distribution of the search pixels among these 8 RAMs is depicted in Fig. 6. All the pixels from the first location of SW are stored into the first location (that is the address 0) of the RAM0. Similarly, all the pixels from the second location of the SW are stored into the first location of the RAM1.

This process is repeated for all the first eight locations of the SW (i.e. for the locations 0 to 7). After the eighth location, the search pixels from the next location (i.e. address 8) are stored into the second location of the RAM0 (i.e. address 1). This process is repeated for the entire SW. This whole process is depicted in Fig. 6 (b). Using the above memory organization, it becomes very easy for one to locate the particular RAM and the location in that RAM where the search pixels from the SW with a given location are stored. For a given location in the SW, the last three bits of the location determine the particular RAM, whereas the first three bits of the location starting from the MSB determine the location of that RAM at which the search pixels with the given location are stored. For example, for the location 7 (000111) of the SW, the first three bits starting from the MSB (i.e. 000) determine the address of the RAM and the last three bits (i.e. 111) determine the particular RAM, i.e. the search pixels from location '7' of the SW are stored at an address 0 of the RAM7. Similarly, the search pixels from the location '47' of the SW are stored at address 5 of RAM7.

C. Register Array for the Current Block Pixels

The pixels from the CB are read only once from the external memory and are stored into an array of sixteen 16-bit registers. The current pixels are then provided to the proper PE by the register array, and no further external memory access is required. In this way, the proposed architecture

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

significantly reduces the number of external memory accesses for the current pixels.

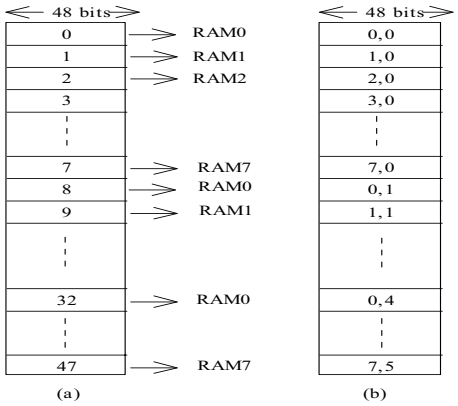


Fig. 6. Memory interleaving for parallel access of the search pixels. (a) Search block for MB of size 16×16 and search range of [-16,16]. (b) Modified memory arrangement with the first index indicating the RAM number and the second indicating the location of 48-bit word stored in the RAM.

D. Search Register Array

In order to provide search pixels to all the PEs simultaneously, an array of five 48-bit registers are used. The data sharing and the data reuse techniques presented in section III are performed by this unit. It reduces the number of external memory accesses to a great extent. At the start of each search, the data are loaded into the register array from the RAMs independently and at the same time. From the next step onwards, no external memory read operations are required for the registers R0-R3 as shown in Fig. 7.

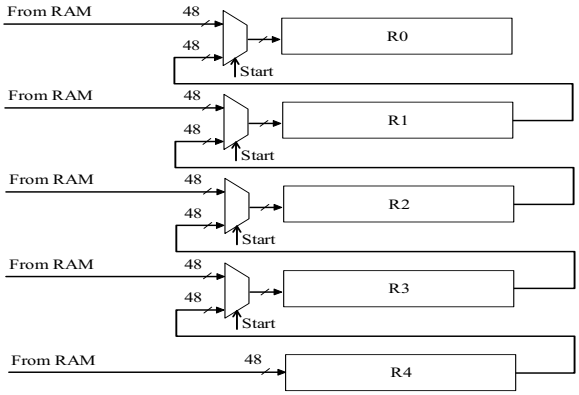


Fig. 7. Register array for search pixels

Register R0 is now connected to register R1 via one multiplexer and register R1 is connected to register R2 via another multiplexer. This is the case for all the registers except for the last register R4, which is always connected with one of the RAMs.

The register R0 always provides the search pixels required for the computation of the NNMP at location '0' (vide Fig. 2). Computation of NNMP at location '0' is done by PE0. Therefore, register R0 is connected to PE0 via one 16-bit data selector. In a similar way, R1 supplies the search pixels required for the computation of NNMP at the locations '1' and '2'. Therefore, register R1 is connected to the processing elements PE1 and PE2 via two 16-bit data selectors. Register R2 is connected to PE3, PE4 and PE5 via three 16-bit data selectors. In a similar way, register R3 is connected to PE6 and PE7, while register R4 is connected to PE8 via data selectors.

The data selector provides the proper data to the proper PE from the register array. It accepts one 48-bit data at its input from the register and provides one 16-bit output to the PE that is connected to it. It selects the proper data depending upon the location of the search center. In particular, if the search center is located at the pixel coordinate (4,4), then one of the data selectors that is attached to register R1 will accept the entire row of 48-bit search data starting from (3,0) to (3,47) from register R1, and will provide one 16-bit search data starting from (3,3) to (3,18) to PE1. Another data selector will provide 16-bit data starting from (3,5) to (3,20) to PE2.

It takes one clock cycle to load the search pixels from the RAMs into the register array and thus it produces a delay of one clock cycle.

E. Comparator Unit

The outputs of all the PEs are connected to a comparator. The comparator selects the best MV from the computed values of the matching criteria. It generates the coordinates of the best matching point.

It also generates a number ranging from 0 to 8 indicating the location where the best match is found. For example, with reference to Fig. 2, if the best match were found at the center of the search, then it will generate the number 4.

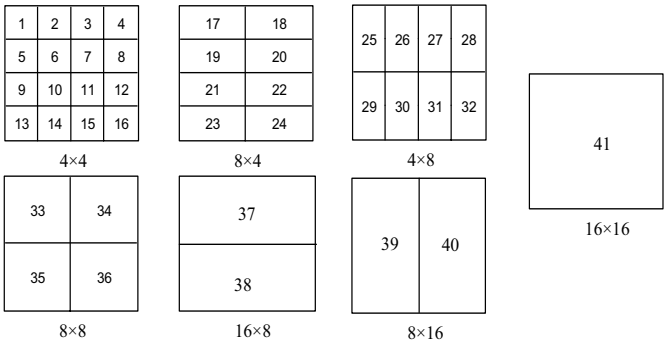


Fig. 8. Partition of a MB into 41 blocks

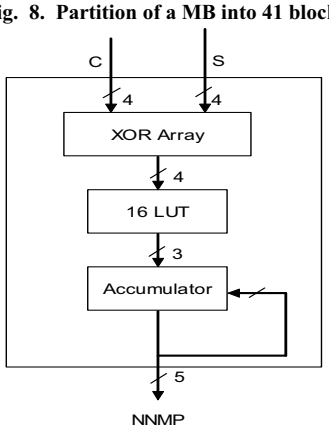


Fig. 9. PE architecture for 1-BT VBS ME

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

F. Process Control Unit

This is the most critical part of the architecture. There are two parts in this unit. One is the pattern analyzer and the other is the control sequence generator.

The pattern analyzer unit takes the decision regarding whether to perform LDSP or SDSP in the next step of the search depending upon the location of the best match as obtained from the comparator unit. It also generates the center of the new search and the new points where the matching criteria are to be evaluated.

The control sequence generator generates the control signals to control the register array for the current block pixels and the search register. It generates the control signals required to coordinate among the different hardware units to complete the search smoothly. It also generates the termination signal indicating that the search process is complete, and the MVs are ready to be read from the MV selector.

V. PROPOSED FAST BINARY ME ARCHITECTURE FOR VARIABLE BLOCK SIZE

The DS based binary ME architecture with VBS is similar to the FBS ME architecture shown in Fig. 4. The VBS motion estimation architecture computes the NNMP values for all the 41 partitions of a MB. The partition of a given MB into 41 blocks is shown in Fig. 8 [9].

However, for VBS ME architecture, an array of 4 PEs is used instead of a single PE as was used for FBS ME architecture. The architecture of a single PE has been shown in Fig. 9. There are two read ports S and C for reading the row of pixels from the SW and the CB respectively. These are then applied at the inputs of the XOR array and the number of 1's as a result of XOR operation is counted by a LUT. The LUT has 16 entries. The output of the LUT is then applied to an accumulator. The final output of the accumulator provides the NNMP value for a particular location.

The detailed structure of PE Array0 has been depicted in Fig. 10. PE Array0 consists of four PEs namely, PE00, PE01, PE02 and PE03. Each PE computes the NNMP value for a primitive block of size 4×4 in 4 clock cycles. One 4-bit counter, which starts counting from the start of the operation (i.e. from time t = 0), is used to keep track of the entire process. The NNMP values for sixteen 4×4 blocks are stored into sixteen 5-bit registers, namely R1 to R16 (vide Fig. 10). The NNMP values for sixteen 4×4 blocks are directed to the appropriate registers by four 1 to 4 demultiplexers. The first two bits starting from the MSB of the counter are used as the control signal for the demultiplexers. Since each PE can compute the NNMP value of a 4×4 block in 4 clock cycles, the four PEs, namely PE00, PE01, PE02 and PE03 work in parallel to produce the NNMP values for the blocks 1, 2, 3 and 4 (vide Fig. 8) respectively at time t= 3. At time t = 3, the counter will read count = 0011 and count [3:2] = 00, so the output of the PE00 will be stored in the register R1. Similarly, the outputs of the other three PEs, namely PE01, PE02 and PE03 will be stored in the registers R2, R3 and R4 respectively.

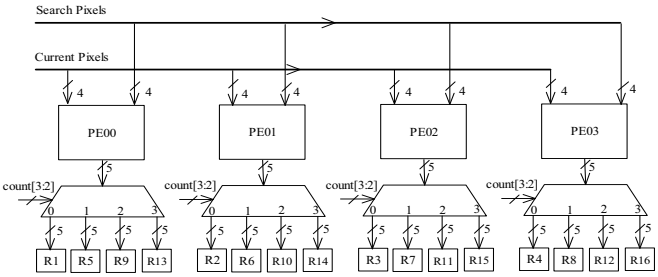


Fig. 10. Configuration of the PE Array0

In this way, all the sixteen NNMP values for sixteen 4×4 blocks are obtained and are stored in the registers R1 to R16. The NNMP values for other block sizes are obtained by adding the NNMP values of the primitive 4×4 blocks. For example, the NNMP values of the two 4×4 blocks are added to obtain the NNMP value for a 4×8 or an 8×4 block. The NNMP values for the 4×8 and 8×4 blocks are stored into a pipeline register [9]. The NNMP values of the two 8×4 blocks are added to obtain the NNMP value of one 8×8 block. The NNMP value of the two 8×8 blocks are added to obtain the NNMP value for one 16×8 or 8×16 block, which are also stored into another pipeline register. The NNMP values of two 8×16 blocks are finally added to obtain the NNMP value for a single MB.

The merging process introduces a delay of 2 clock cycles for two stage pipelining. The 41 NNMP values computed for a MB are finally sent to a comparator and a MV selector, which

VI. DETERMINE THE MINIMUM NNMP VALUE AND THE CORRESPONDING MVS FOR EACH MB PARTITION. RESULTS

A. Performance of the Proposed Fast 1-BT Based ME

The PSNR difference for the proposed method of ME has been compared with the conventional FS based binary ME. The results have been shown in Table III. It can be observed from Table III that the proposed method of ME can provide a very small PSNR degradation compared to the FS based binary ME. For a video sequence with low motion like Hall Monitor, the PSNR drop is below 0.1 dB. However, the PSNR drop increases slightly for the sequences with complex motion like Foreman and Coastguard.

The FS based binary ME has been compared with the proposed method of ME in terms of the average number of steps required to process a MB, and the result of the comparison has been provided in Table IV.

Based on the observations made on Table III and Table IV, it can be concluded that the computational complexity of the proposed method is much lower than the FS based ME. At the same time, the degradation in PSNR is also less than 0.5 dB for all the video sequences tested.

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

TABLE III
AVERAGE PSNR DROP (DB) AGAINST CONVENTIONAL FULL SEARCH BASED BINARY ME

Foreman	Hall Monitor	Football	Tennis	Coastguard
0.308	0.080	0.191	0.209	0.478

TABLE IV
COMPARISON OF THE AVERAGE NUMBER OF SEARCH STEPS REQUIRED TO PROCESS ONE MB FOR DS (FOR FS IT IS ALWAYS 1089)

Video Sequence	Average no. of Search Steps
Foreman	6.85
Hall Monitor	3.76
Football	4.92
Tennis	5.62
Coastguard	3.98

B. Implementation Results

The proposed DS based binary ME architectures for FBS and VBS are described in Verilog HDL. In order to make a fair comparison with the architectures reported in [9], the proposed architectures are also implemented on the same FPGA as was done in [9].

The proposed fast binary FBS ME architecture consumes 731 slices (1103 LUTs). The on-chip memory of the proposed architecture is 2304 bits for storing the SW for a single MB. These search pixels are stored into 6 locations of 8 RAMs. The proposed VBS ME architecture consumes 1014 slices (1576 LUTs).

The proposed VBS ME architecture involves a latency of 22 clock cycles: one clock cycle for loading the search pixels from the RAMs into the register array for search pixels, 16 clock cycles required for the computation of the NNMP, two clock cycles for the merging process, one clock cycle for the comparison and one clock cycle each required for the purpose of the pattern generation and the control signal generation. Therefore, each of the steps of DS requires 22 clock cycles. Based on our simulation results (vide Table IV), it is found that when applying DS on 1-BT frames, a maximum of 7 search steps are required for processing a single MB. Thus, a total 154 (22×7) clock cycles are required to process a single MB.

Comparison of the proposed architecture with various other architectures is presented in Table V. It then follows from Table V that the proposed architecture involves the least number of clock cycles required for processing a single MB. One major advantage of having the least number of clock cycles requirement for processing one MB is that the clock frequency required to process a video sequence with a given frame size, and the frame rate is substantially reduced compared to other architectures. The reduction of the minimum clock frequency requirement has a direct impact on reducing the overall power consumption for ME hardware [16]. The clock frequencies required for the proposed VBS ME architecture and its counterparts ([9], [10]) for processing different video sequences with different frame sizes and rates have been provided in Table VI. It can be observed from Table VI that for the proposed architecture, a clock frequency of 16.63 MHz is sufficient to perform VBS ME for a video sequence with SDTV frame size (1280×720) and a frame rate of 30 fps. On the other hand, the required clock frequencies are 30.46 MHz and 112.22 MHz for the architectures [9] and [10] respectively for processing the same video sequence.

TABLE VI
REQUIRED CLOCK FREQUENCIES FOR THE PROPOSED ARCHITECTURE AND OTHER 1-BT ME ARCHITECTURES FOR PROCESSING DIFFERENT VIDEO SEQUENCES

Frame size		CIF (352×288)	SDTV (1280×720)	HDTV (1280×720)	SHDTV (1920×1080)
Frame rate (fps)		30	30	60	60
Clock rate (MHz)	Proposed	1.82	16.63	33.26	74.84
	Akin [9]	3.36	30.46	60.91	137.05
	Celebi [10]	12.34	112.22	224.41	504.95

Accordingly, the power consumption for processing the SDTV frame is only 59 mW for the proposed architecture, whereas the power consumptions are 83 mW and 96 mW for the architectures [9] and [10] respectively.

VII. CONCLUSIONS

In the present paper, low power ME architectures have been developed for implementing DS on 1-BT frames with FBS and VBS support. As compared with other 1-BT based ME architectures, the proposed architectures involve the lowest latency. The clock frequency of the proposed ME architectures therefore can be effectively reduced for low

TABLE V
COMPARISON WITH OTHER 1-BT BASED ME ARCHITECTURES

	Proposed (FBS)	Proposed (VBS)	[9] (FBS)	[9] (VBS)	[10]
No. of PEs/MB	9	36 (9×4)	256	256	16
On-chip memory	2304	2304	4608	4608	24064
Supported block sizes	16×16	4×4 to 16×16	16×16	4×4 to 16×16	16×16
Search range	[-16,16]	[-16,16]	[-16,16]	[-16,16]	[-16,15]
No. of clock cycles/MB	140	154	282	282	1039
Area	731 slices (1103 LUTs)	1014 slices (1576 LUTs)	4758 slices (7280 LUTs)	6782 slices (8702 LUTs)	944 slices (1467 LUTs)
Maximum Freq. (MHz)	135	129	115	113	127

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.

power designs. In particular, the required clock frequency for the proposed VBS ME architecture for processing SDTV frames (1280×720 @ 30 fps) is 16.63 MHz. The resulting power dissipation is only 59 mW, which may be an attractive solution for portable consumer video applications typically operated by battery power. On the other hand, the frame size and the frame rate supported by the proposed architectures can also be extended subject to a clock frequency constraint. For the proposed VBS ME architecture, the required clock frequency for processing SHDTV (1920×1080 @ 60 fps) is 74.84 MHz. The maximum operating frequencies of the proposed architectures for FPGA implementation are found to be 135 MHz and 129 MHz for FBS and VBS respectively. The proposed architectures are therefore deemed suitable for designing consumer electronic products that require real time video processing or compression at affordable prices.

REFERENCES

[1] Z. L. He, C. Y. Tsui, K. K. Chan, and M. L. Liou, "Low-power VLSI design for motion estimation using adaptive pixel truncation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 10, no. 5, pp. 669-678, Aug. 2000.

[2] C. H. Hsieh and T. P. Lin, "VLSI architecture for block-matching motion estimation algorithm," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 2, no. 2, pp. 169-175, June 1992.

[3] H. Gharavi and M. Mills, "Blockmatching motion estimation algorithms-new results," *IEEE Trans. on Circuits and Systems*, Vol. 37, no. 5, pp. 649-665, May 1990.

[4] B. Natarajan, V. Bhaskaran, and K. Konstantinides, "Low-complexity block-based motion estimation via one-bit transforms," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 7, no. 3, pp. 702–706, Aug. 1997.

[5] S. Ertürk, "Multiplication-free one-bit transform for low-complexity block-based motion estimation," *IEEE Signal Process. Lett.*, vol. 14, no. 2, pp. 109–112, Feb. 2007.

[6] H. Lee and J. Jeong, "Early termination scheme for binary block motion estimation," *IEEE Trans. Consumer Electron.*, vol. 53, no. 4, pp. 1682–1686, Nov. 2007.

[7] A. Ertürk and S. Ertürk, "Two-bit transform for binary block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 15, no. 7, pp. 938–946, Jul. 2005.

[8] O. Urhan and S. Ertürk, "Constrained one-bit transform for low complexity block motion estimation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 17, no. 4, pp. 478–482, Apr. 2007.

[9] A. Akin, Y. Dogan, and I. Hamzaoglu, "High performance hardware architectures for one bit transform based motion estimation," *IEEE Trans. on Consumer Electron*., vol. 55, no. 2, pp. 941-949, May 2009.

[10] A. Celebi, O. Urhan, I. Hamzaoglu, and S. Ertürk, "Efficient hardware implementation of low bit depth motion estimation algorithms," *IEEE Signal Processing Lett.*, vol. 16, no. 6, pp. 513-516, June 2009.

[11] S. Zhu and K. K. Ma, "A new diamond search algorithm for fast block-matching motion estimation", *IEEE Trans. Image Processing*, vol. 9, no. 2, pp. 287-290, Feb. 2000.

[12] E. S. Lee, O. Urhan, and T. G. Chang, "Multiplication-free one-bit transform and diamond search combination for fast binary block motion estimation," in *Proc. 15th International Conference on Signal Processing and Communications Applications*, SIU-2007.

[13] T. C. Chen, Y. H. Chen, S. F. Tsai, S. Y. Chien, and L. G. Chen, "Fast algorithm and architecture design of low-power integer motion estimation for H.264/AVC," *IEEE Trans. Circuits Syst. Video Technol*., vol. 17, no. 5, pp. 568–577, May 2007.

[14] D. Ding, S. Yao, and L. Yu, "Memory bandwidth efficient hardware architecture for AVS encoder," *IEEE Trans. on Consumer Electron*., vol. 54, no. 2, pp. 675–680, May 2008.

[15] C. Wei, H. Hui, T. Jiarong, and M. Hao, "A high-performance reconfigurable VLSI architecture for VBSME in H.264," *IEEE Trans. Consumer Electron.*, vol. 54, no. 3, pp. 1338–1345, Aug. 2008.

[16] C. M. Ou, C. F. Le, and W. J. Hwang, "An efficient VLSI architecture for H.264 variable block size motion estimation," *IEEE Trans. on Consumer Electron*., vol. 51, no. 4, pp. 1291–1299, Nov. 2005.

[17] M. Parlak and I. Hamzaoglu, "Low power H.264 deblocking filter hardware implementations," in *Proc Second NASA/ESA Conference on Adaptive Hardware and Systems*, (AHS-2007).

[18] P. Kuhn, *Algorithms, Complexity Analysis and VLSI Architectures for MPEG-4 Motion Estimation*, 1st ed., Kluwer Academic Press, 1999, pp. 120-128.

[19] H. M. Jong, L. G. Chen, and T. D. Chiueh, "Parallel architectures for 3-step hierarchical search block-matching algorithm," *IEEE Trans. on Circuit Syst. Video Technol.*, Vol. 4. No. 4, pp. 407-416, August 1994.

BIOGRAPHIES



Sumit K Chatterjee is a PhD student at the Department of Electronics and Electrical Communication Engineering, Indian Institute of Technology, Kharagpur, India. He received his M.Sc. degree in Physics from Banaras Hindu University, Varanasi, India. He is having more than ten years of teaching and research experiences. His research interests include video processing and VLSI architecture development in the field of video processing.



Indrajit Chakrabarti received the Ph.D from Indian Institute of Technology (IIT) Kharagpur, India in 1997. From 1998 to 2004, he worked as an Assistant Professor and later as an Associate Professor in the Department of Electronics and Communication Engineering, IIT Guwahati. He is presently serving as an Associate Professor in the Department of Electronics and Communication Engineering, IIT Kharagpur. His areas of interest include VLSI architectures for image and video processing, digital signal processing and telecommunication.

Authorized licensed use limited to: University Of Minnesota Duluth. Downloaded on October 11,2026 at 20:21:35 UTC from IEEE Xplore. Restrictions apply.